needle: Cost-Aware Credit Card Fraud Detection under Extreme Class Imbalance

Davide Carella
Università degli Studi di Bari Aldo Moro
Italy
d.carella12@studenti.uniba.it

***Abstract*—Credit card fraud detection is a canonical example of a machine learning problem where predictive accuracy is worthless: in a dataset where 0.17% of transactions are fraudulent, a model that never predicts fraud is 99.83% accurate. This project treats the imbalance itself as the central technical challenge rather than a preprocessing detail.**

We will use the ULB Credit Card Fraud Detection dataset (284,807 transactions over two days, of which 492 are fraudulent, with 28 anonymized PCA features plus transaction time and amount). The work proceeds in four stages. First, metric selection: we will evaluate models with precision-recall AUC and recall at a fixed precision or at a fixed number of daily alerts. ROC-AUC will be reported only as a secondary number, since with 578 negatives per positive it obscures large changes in the false positive rate. Second, a validation design that guards against leakage: exact duplicates are removed before splitting, all resampling and scaling is done inside cross-validation folds, and a final temporal holdout (train on day one, test on day two) estimates performance after a distribution change. Third, a comparison across a ladder of models: a trivial baseline, regularized logistic regression, tree ensembles, gradient boosting tuned against PR-AUC, and an unsupervised anomaly detector trained on legitimate transactions only crossed with three imbalance strategies (class weighting, undersampling, SMOTE). Fourth, explicit threshold selection and calibration, with a cost model that weights missed frauds by transaction amount against a fixed review cost per false alarm.

Deliverables are a reproducible codebase, a precision-recall analysis with the chosen operating point justified in cost terms, SHAP-based interpretation of the winning model, and a discussion of limitations.

***Index Terms*—Class imbalance, Cost-sensitive learning, Credit card fraud detection, Model calibration, Precision-recall analysis, Threshold selection**

I. Introduction

Going from a simple classifier that emits a label to a production-ready fraud detector is not easy. Every methodological question here tries to remove the gap between the two: which metric to trust when accuracy is a constant, how to split so the score survives a change of day, when to cut the ranking when the analysts are finite, and what a missed fraud costs against a wasted review.

The dataset [1] used is two days of European card transactions: 284,807 rows, 492 frauds, 28 anonymised PCA components plus a time and an amount. ROC-AUC is nearly as uninformative as accuracy, because its abscissa divides false alarms by 284,315 negatives, so the whole span between a model an analyst can use and one that buries them fits in the leftmost few percent of the curve.

This report documents `needle`, a command-line pipeline that ships an operating point from the raw dataset in one reproducible run; a significance test shows that the top models do not separate.

II. Related work

A. The dataset and its authors

The data comes from a collaboration between Worldline and the Machine Learning Group at the Université Libre de Bruxelles. Dal Pozzolo et al. [2] confirm the approach adopted here: fraud detection as alert generation under a bounded review budget, and [3] adds verification latency and concept drift, which a static extract cannot express. [4] shows analytically that undersampling shifts the posterior; it is why we implement calibration at all. Later work in the same line addresses streaming and scale [5], active learning under a labelling budget [6], domain adaptation [7] and incremental retraining [8]; [9] treats the unsupervised branch, combining outlier scores with a supervised model rather than choosing between them. [10] and the thesis [11] are the group's fullest statements of method.

B. Evaluation under imbalance

Davis and Goadrich [12] established that ROC and PR space weight the same errors very differently; Saito and Rehmsmeier [13] made the case directly for skewed data; Elkan [14] supplies the decision-theoretic basis for choosing a threshold from costs rather than from a metric.

C. Resampling and class weighting

SMOTE [15] is one of three strategies that `needle` can use to resample transactions. [16] finds its benefit largely vanishes in high dimensions where the minority class is not locally dense, and [17] finds that for strong ensembles it rarely beats class weighting once the threshold is tuned. Our results will report the opposite.

D. Leakage

Hayat and Magnier [18] examine published methodology around this dataset and identify four recurring failures: leakage from preprocessing before splitting, vague reporting, absent temporal validation, and recall optimised at precision's

expense. Every element answers one of them, and the protocol was fixed before the models were compared.

III. Methodology

A. Data and features

The data loader removes 1,081 exact duplicate rows, 19 of which are frauds. Doing this is very important because an identical row may land on both sides of a split, resulting in leakage.

The following operations are applied on the features of the dataset:

- `V1-V28` are untouched, they result from PCA and thus are already scaled and uncorrelated;
- `Amount` is replaced with $\text{log\_amount} = \log(1 + \text{Amount})$ to reduce the effect of its heavy tail and, finally, a `RobustScaler` is applied to scale the feature while reducing the impact of outliers;
- `Time` is replaced with $\text{hour} = (\text{Time}/3600) \bmod 24$ to capture the diurnal signal.

B. The metrics used

PR-AUC is the primary metric used to evaluate the models, as it is sensitive to the minority class. Recall at precision ≥ 0.90 is also reported, as it reflects the operational needs of a review team. The deciding metric is an amount-weighted cost, for a threshold t:

$$\begin{aligned}\text{cost}(t) = &\sum_{i:y_i=1, s_i<t} a_i \\ &+ c_{\text{review}} \cdot |\{i : y_i = 0, s_i \geq t\}|\end{aligned} \tag{1}$$

where a_i is the amount, s_i is the score given to the transaction, and $c_{\text{review}} = 3$ is the cost of reviewing a transaction.

ROC-AUC is still reported for comparability, but it is not informative in this context, as it does not reflect the operational constraints of the review team. The abscissa of the ROC curve divides false positives by all negatives. This means that a model with a high ROC-AUC may still generate an unmanageable number of false positives for the review team.

C. Data split

After deduplication and before anything is fitted, the dataset is split temporally into day 1 (144,236 rows, 272 frauds) and day 2 (139,490 rows, 201 frauds). Day 1 is used for hyperparameter search, model selection and threshold choice; day 2 is used only for the final evaluation of the selected model.

The hyperparameter search uses a stratified 5-fold cross-validation on day 1, while model selection uses a repeated stratified 5-fold cross-validation with 3 repeats (15 folds in total) on day 1. The threshold is chosen based on the out-of-fold scores from the model selection step.

D. Models, imbalance and tuning

We experimented with different supervised models:

- a stratified dummy as a baseline;
- logistic regression;
- random forest [19];
- LightGBM [20].

Each of them was combined with class weighting, random undersampling and SMOTE. Every candidate is a scikit-learn [21] estimator wrapped in an `imblearn.Pipeline` [22], so scaling and resampling are fitted on the training part of each fold only and never on the fold being scored.

Unsupervised models were also tested, in particular an isolation forest [23] and a denoising autoencoder trained on legitimate transactions only, to answer whether labels are needed at all.

Optuna [24] with a TPE sampler [25] maximises PR-AUC over five folds, with budgets of 60 trials for LightGBM, because of its large amount of parameters to tune, and 15-20 for the others. Every tuned winner is re-scored on the full repeated stratified 5-fold CV before entering the leaderboard, to correct for selection bias.

E. Statistical significance

The final leaderboard of the pipelines by itself doesn't tell us if the top model is actually better than the others, or if the differences are just due to random chance. To answer this question, we perform a corrected resampled t-test [26] on the PR-AUC scores of the top model against the five named challengers, read at a significance level of $\alpha = 0.05$ fixed before the comparison. The p-value is also Holm adjusted [27], because testing one winner against five challengers on a single set of folds is five chances at a gap that is not there: at that level the probability of at least one false positive somewhere in the family is $1 - 0.95^5 = 22.6\%$, not 5%.

F. Threshold, calibration and interpretation

To make the ranking of a model a decision, we need to decide on a threshold. The scores are sorted once by descending score and the sweep keeps one entry per distinct score, thus collapsing tied blocks. Frauds caught, queue length, and all other metrics are then prefix sums at those boundaries, so the full sweep costs one $O(n \log n)$ sort and a constant number of linear passes instead of a rescan per threshold.

Three different objectives are computed each run:

- `cost`: the minimiser of (1);
- `precision`: the highest recall that clears a 0.90 precision floor;
- `budget`: the largest queue inside a stated capacity.

Every infeasible request is flagged rather than silently approximated and, because c_{review} is an assumption, the optimum is recomputed across $c_{\text{review}} \in \{1, 3, 5, 10, 30, 100\}$ every run.

Resampling breaks the meaning of a score [4], so the winner is also fitted inside a `CalibratedClassifierCV` with sigmoid scaling [28], rather than isotonic because 272 positives cannot support a monotone map.

Interpretation uses SHAP [29] with the exact tree explainer [30] over all day 2 frauds plus 4,000 sampled legitimate rows; the fraud over-representation is deliberate, so the ranking answers "what separates the classes".

IV. Results

A. The leaderboard

TABLE I
All seventeen candidates.

Pipeline	PR-AUC	±	R@P.90	ROC-AUC
`lgbm/smote` (tuned)	**.8783**	.0347	.8593	.9754
`rf/smote` (tuned)	.8747	.0321	.8555	.9777
`rf/smote`	.8697	.0326	.8397	.9731
`rf/weighted`	.8618	.0351	.8470	.9551
`lgbm/smote`	.8236	.0544	.8262	.9633
`lr/none` (tuned)	.7355	.0559	.3682	.9543
`rf/under`	.7139	.0584	.2497	.9714
`lr/smote`	.7031	.0535	.1931	.9680
`lr/weighted`	.6943	.0558	.1783	.9675
`lgbm/under`	.6696	.0730	.1731	.9716
`lgbm/weighted`	.6660	.0701	.3407	.9211
`autoencoder/none`	.5298	.0961	.1694	.9412
`lr/under`	.5224	.1372	.0833	.9661
`autoencoder/none` (tuned)	.5129	.1224	.1166	.9440
`isolation_forest/none` (tuned)	.4237	.0653	.0798	.9512
`isolation_forest/none`	.2306	.0608	.0000	.9453
`dummy/none`	.0019	.0001	.0000	.4997

As we anticipated, ROC-AUC and PR-AUC disagree on the order and scale of the ranking, ROC-AUC spans 0.9211-0.9777 while PR-AUC spans 0.2306-0.8783, worse than that the isolation forest scores a higher ROC-AUC than the weighted LightGBM but has a third of the PR-AUC. Ranking on ROC-AUC would ship an unusable model over a working one.

Another important finding is that the imbalance strategy matters more than the learner, within LightGBM the three strategies span 0.158 PR-AUC (0.666 weighted, 0.670 under, 0.824 SMOTE). Moreover, in contrast with the literature, SMOTE beat weighting here, at this seed, for tree ensembles tuned jointly with the choice.

Finally, labels are very important: the best unsupervised detector reaches 0.5298 against the winner's 0.8783.

B. Significance testing

TABLE II
Winner vs. the five named challengers over 15 paired folds.

Challenger	Δ	t	p uncorrected	p corrected
`rf/smote` (tuned)	.0036	0.63	.189	.537
`rf/smote`	.0086	1.66	.0029	.240
`rf/weighted`	.0165	2.20	.00028	.180
`lgbm/smote`	.0547	2.09	.00045	.180
`lr/none` (tuned)	.1428	7.49	1.7e−10	1.5e−5

The first four rows are above α, so on these folds we cannot tell whether the winner is better than them: each gap fits inside the spread between folds. The last row is below α and its Δ is positive, so the winner is better than the tuned logistic regression, by 0.14 PR-AUC against a fold spread of 0.035.

So the order of the top five rows of Table I is not supported by the data. The pipeline carried to day 2 is still the top of that leaderboard, a choice made before the test was run: the test does not overturn it, it only says that the four rows beneath would have served as well. Breaking the tie on something the test does not measure would be the better engineering rule, and it is a different decision from the one taken here. On fit cost it would have shipped the untuned `lgbm/smote`, which trains in 0.99 s against the winner's 10.9 s.

C. Testing against the second day

The same pipeline scores PR-AUC 0.8783 ± 0.0347 in day-1 CV and 0.7644 on day 2. Recall at the precision floor falls from 0.8593 to 0.6517. ROC-AUC does the opposite and rises, from 0.9754 to 0.9790.

The drop has three causes, and only one of them is a fault of the model:

- random folds are too easy, because fraud arrives in bursts on the same stolen card, so two frauds are often almost the same row and a random split puts one in train and the other in test;
- the share of frauds changed, from 0.189% on day 1 to 0.144% on day 2, a quarter fewer, and PR-AUC depends on that share, so part of the drop is just arithmetic;
- the rest is real drift, since day 2 transactions do not follow day 1.

Two days give only one split, so we cannot measure how much each cause contributes. What we can say is that 0.764 is the number to expect on a new day, and 0.878 describes day 1 alone.

D. The operating point and its transfer

TABLE III
OPERATING POINTS. ROWS 1–3 ARE THE THREE OBJECTIVES ON DAY-1 OUT-OF-FOLD SCORES; ROWS 4–5 CARRY THE SHIPPED `cost` THRESHOLD (0.016896) TO DAY 2; ROW 6 IS DAY 2'S OWN MINIMUM.

Point	Alerts	Prec.	Rec.	Cost
Day 1 OOF, `cost`	524	.4637	.8934	4,234
Day 1 OOF, `precision`	253	.9170	.8529	5,301
Day 1 OOF, `budget`	100	1.000	.3676	22,217
Day 2, same threshold	2,551	.0670	.8507	15,712
Day 2, same alert rate	507	.3195	.8060	10,680
Day 2 optimum (hindsight)	217	.7235	.7811	9,871

From the table we can see that the number of reviews we can handle changes everything, with only 100 alerts a day every alert is fraud but we catch only 36.8% of it, thus costing us 5.2 times the best point. We can also see that on the second day, with the same threshold, the recall stays high but we waste much more work thus increasing the cost by 3.7 times. Also, neither of the ways of moving the threshold to day 2 is ideal, but one is much better: keeping the old alert rate costs 8% more than the day-2 optimum, while keeping the old threshold costs 59% more.

Recall counts frauds, not money. At the shipped threshold the day-2 queue holds 2,551 alerts, 171 of them fraud, and the 30 frauds that get through (Fig. 1) are 14.9% of the count but 32.8% of the fraudulent amount: they average 285.72 against 102.64 for the ones caught, and the largest is 2,125.87. A recall of 0.85 therefore recovers 67.2% of the money at risk, so reading recall as value recovered is optimistic on this data.

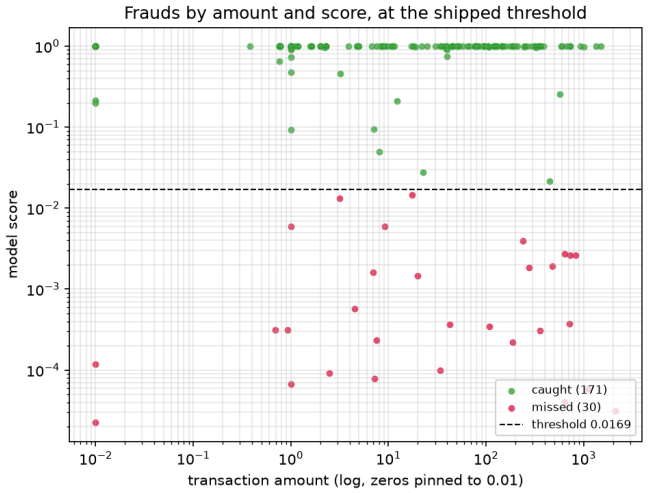

Fig. 1. Every day-2 fraud by amount and score. Green is caught at the shipped threshold, red is missed. The misses are not the cheap frauds, and most of them score far enough below the line that no small move of the threshold reaches them.

The price of a review is an assumption, so the cost optimum is recomputed over the grid named in Section III.F. A hundredfold rise in c_{review} moves the threshold from 0.0169 to 0.9919, but the queue only shrinks from 524 alerts to 233 and recall only falls from 0.893 to 0.824, while precision rises from 0.46 to 0.96. The decision is far more stable in alert volume than in the cut-off that produces it, which is a second reason to state an operating point as a queue length.

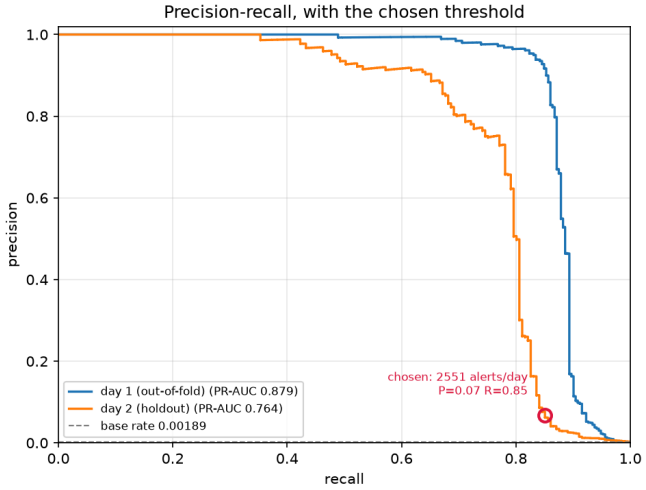

Fig. 2. Day-1 out-of-fold against the day-2 holdout. The ring is the shipped point as it landed on day 2.

E. Interpretation

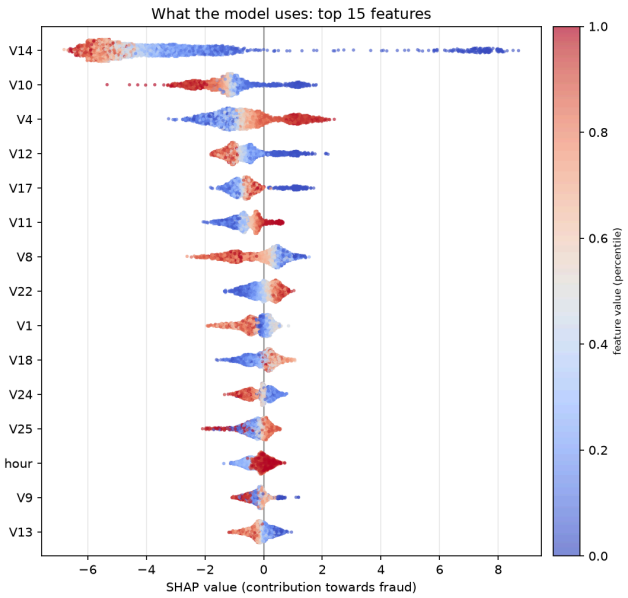

Fig. 3. SHAP values over all day-2 frauds plus 4,000 sampled legitimate rows. Every dot is one transaction, its position on the x axis is how much the feature pushed the score towards fraud, and its colour is the value of the feature, blue for low and red for high.

SHAP tells us which features the model actually uses. `V14` alone carries 33.5% of the total attribution and the top five features carry 62.4%, so the decision rests on a handful of features. This is good because few features are easier to audit.

This also shows the direction of each feature. For `V14` the low values, in blue, are the ones that push the score towards

fraud, and they push it much further than any other feature does. The two features we built ourselves earn a modest place: `hour` is 13th with 1.7% and `log_amount` is 16th with 1.3%, below fourteen of the PCA components. Since `V1`-`V28` come from a transform that was never published, we can only say which features matter, not what they mean.

F. Calibration

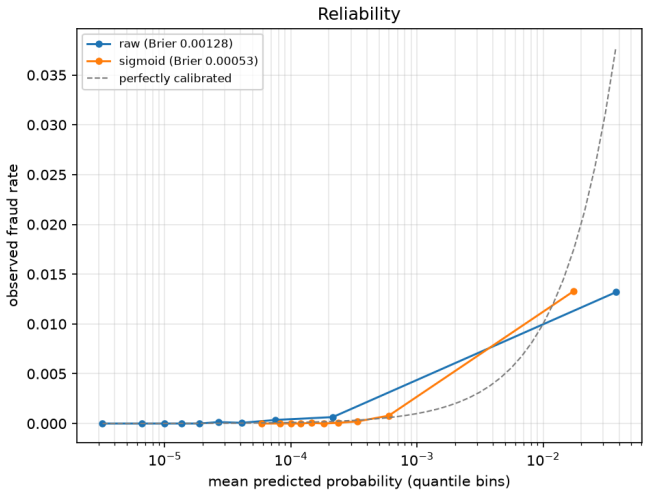

Fig. 4. Predicted probability against the fraud rate we really observed on day 2. The dashed line is what a perfect model would draw. The x axis is logarithmic because almost every score is very small.

The threshold only needs the order of the scores, but as soon as we read a score as a probability it has to match reality. Resampling breaks this, because SMOTE trains the model on a set where frauds are far more common than they really are [4].

Fig. 4 shows the problem. The raw curve sits below the dashed line, so the model claims more fraud than actually happens. Its mean score is 0.003821 against a real rate of 0.001441, so it overstates the risk by 2.7 times. Sigmoid calibration moves the curve closer to the line and the mean score down to 0.001918, and the Brier score falls from 0.001283 to 0.000526, less than half of what it was.

V. Discussion and limitations

This project confirms what was already supported, that ROC-AUC is the wrong metric to use to select a model, a frozen threshold does not preserve its alert volume, and that the imbalance strategy outweighs the learner. It also shows that recall overstates value recovery, and that review capacity binds before model quality does. The project also shows that SMOTE beat weighting on this data and seed, but this is not a general result.

The main limitation is the extreme scarcity of frauds: 473 in total, 272 of them trainable. This is why the CV is repeated, why the fold spread is quoted with every mean, why the significance test needs a variance correction, why the search bought variance reduction over capacity, and why calibration is sigmoid. Other limitations are: number of days, two days give only one temporal split; anonymised features rule out domain checks and make the expensive misses undiagnosable; the cost model is an assumption, pricing a miss at the full amount and a review at a flat 3, ignoring recovery, chargeback fees and wrongly-blocked customers. Finally, the unsupervised branch is under-explored, [9] finds value in combining outlier scores with a supervised model rather than choosing, which was not tested here. Verification latency is ignored entirely.

VI. Reproducibility

The full source code and instructions to reproduce the run are in the `needle` repository [31]. All the experiments described were run on an AMD Ryzen 5 5600X and 16 GiB RAM, at seed 42 throughout. The shipped LightGBM uses:

- `n_estimators` 1400;
- `learning_rate` 0.0151;
- `num_leaves` 96;
- `min_child_samples` 145;
- `subsample` 0.524 at `subsample_freq` 1;
- `colsample_bytree` 0.829;
- `reg_alpha` 1.5×10^{-4};
- `reg_lambda` 2.30;
- default SMOTE sampling ratio (1).

The dataset can be downloaded from the Kaggle [1].

VII. Conclusions

At a 0.167% positive rate ROC-AUC cannot separate a deployable ranker from an unusable one, it varied eleven times less than PR-AUC across sixteen candidates (excluding the dummy), inverted their order outright, and improved across the split on which precision fell sevenfold. A tuned LightGBM over SMOTE reached 0.878 ± 0.035 in CV and 0.764 on the holdout, but the corrected paired test could not separate it from four challengers where the uncorrected test would have separated four, so what ships is the top of the leaderboard rather than a winner the evidence singles out, and the resampling strategy proved worth more than the learner. Carried to day 2 the threshold held recall and lost precision from 0.46 to 0.07, with day 2's own optimum at 217 alerts against 2,551 raised, so alert volume rather than a probability cut-off is the quantity to specify.

References

[1] Machine Learning Group, Université Libre de Bruxelles and Worldline, "Credit Card Fraud Detection." [Online]. Available: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

[2] A. Dal Pozzolo, O. Caelen, Y.-A. Le Borgne, S. Waterschoot, and G. Bontempi, "Learned lessons in credit card fraud detection from a practitioner perspective," *Expert Systems with Applications*, vol. 41, no. 10, pp. 4915–4928, 2014, doi: 10.1016/j.eswa.2014.02.026.

[3] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 8, pp. 3784–3797, 2018, doi: 10.1109/TNNLS.2017.2736643.

[4] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating Probability with Undersampling for Unbalanced Classification," in *2015 IEEE Symposium Series on Computational Intelligence*, IEEE, 2015, pp. 159–166. doi: 10.1109/SSCI.2015.33.

[5] F. Carcillo, A. Dal Pozzolo, Y.-A. Le Borgne, O. Caelen, Y. Mazzer, and G. Bontempi, "SCARFF: A scalable framework for streaming credit card fraud detection with Spark," *Information Fusion*, vol. 41, pp. 182–194, 2018, doi: 10.1016/j.inffus.2017.09.005.

[6] F. Carcillo, Y.-A. Le Borgne, O. Caelen, and G. Bontempi, "Streaming active learning strategies for real-life credit card fraud detection: assessment and visualization," *International Journal of Data Science and Analytics*, vol. 5, no. 4, pp. 285–300, 2018, doi: 10.1007/s41060-018-0116-z.

[7] B. Lebichot, Y.-A. Le Borgne, L. He-Guelton, F. Oblé, and G. Bontempi, "Deep-Learning Domain Adaptation Techniques for Credit Cards Fraud Detection," in *Recent Advances in Big Data and Deep Learning (INNSBDDL 2019)*, Springer International Publishing, 2019, pp. 78–88. doi: 10.1007/978-3-030-16841-4_8.

[8] B. Lebichot, G. M. Paldino, W. Siblini, L. He-Guelton, F. Oblé, and G. Bontempi, "Incremental learning strategies for credit cards fraud detection," *International Journal of Data Science and Analytics*, vol. 12, no. 2, pp. 165–174, 2021, doi: 10.1007/s41060-021-00258-0.

[9] F. Carcillo, Y.-A. Le Borgne, O. Caelen, Y. Kessaci, F. Oblé, and G. Bontempi, "Combining unsupervised and supervised learning in credit card fraud detection," *Information Sciences*, vol. 557, pp. 317–331, 2021, doi: 10.1016/j.ins.2019.05.042.

[10] Y.-A. Le Borgne, W. Siblini, B. Lebichot, and G. Bontempi, *Reproducible Machine Learning for Credit Card Fraud Detection — Practical Handbook*. Université Libre de Bruxelles, 2022. [Online]. Available: https://github.com/Fraud-Detection-Handbook/fraud-detection-handbook

[11] A. Dal Pozzolo, "Adaptive Machine Learning for Credit Card Fraud Detection," Doctoral dissertation, 2015.

[12] J. Davis and M. Goadrich, "The relationship between Precision-Recall and ROC curves," in *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, 2006, pp. 233–240. doi: 10.1145/1143844.1143874.

[13] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLOS ONE*, vol. 10, no. 3, p. e0118432, 2015, doi: 10.1371/journal.pone.0118432.

[14] C. Elkan, "The Foundations of Cost-Sensitive Learning," in *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, 2001, pp. 973–978.

[15] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002, doi: 10.1613/jair.953.

[16] R. Blagus and L. Lusa, "SMOTE for high-dimensional class-imbalanced data," *BMC Bioinformatics*, vol. 14, p. 106, 2013, doi: 10.1186/1471-2105-14-106.

[17] Y. Elor and H. Averbuch-Elor, "To SMOTE, or not to SMOTE?." [Online]. Available: https://arxiv.org/abs/2201.08528

[18] K. Hayat and B. Magnier, "Data Leakage and Deceptive Performance: A Critical Examination of Credit Card Fraud Detection Methodologies." [Online]. Available: https://arxiv.org/abs/2506.02703

[19] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[20] G. Ke *et al.*, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017, pp. 3146–3154.

[21] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[22] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning," *Journal of Machine Learning Research*, vol. 18, no. 17, pp. 1–5, 2017.

[23] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *2008 Eighth IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422. doi: 10.1109/ICDM.2008.17.

[24] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631. doi: 10.1145/3292500.3330701.

[25] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for Hyper-Parameter Optimization," in *Advances in Neural Information Processing Systems 24 (NIPS)*, 2011, pp. 2546–2554.

[26] C. Nadeau and Y. Bengio, "Inference for the Generalization Error," *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003, doi: 10.1023/A:1024068626366.

[27] S. Holm, "A Simple Sequentially Rejective Multiple Test Procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[28] J. C. Platt, "Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods," in *Advances in Large Margin Classifiers*, MIT Press, 1999, pp. 61–74.

[29] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017, pp. 4765–4774.

[30] S. M. Lundberg *et al.*, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020, doi: 10.1038/s42256-019-0138-9.

[31] D. Carella, "needle: Cost-Aware Credit Card Fraud Detection under Extreme Class Imbalance." [Online]. Available: https://github.com/ITHackerstein/needle